

Ingeniería Agéntica

Sesión 1 — Fundamentos y Vibe Coding

Contenidos de hoy

01 El curso

02 Fundamentos de los LLMs

03 Cómo se usa un modelo

04 Agentes

05 Vibe coding

06 Práctica

O I

El curso

Sobre nosotros

PROFES

Diego Piloni — Agustín Carrasco

INVITADO

Alejandro Silva (CCAD — UNC)

Sobre ustedes

- ¿Quiénes son?
- ¿Estudian? ¿Trabajan?
- ¿Usan IA? ¿Para qué?
- ¿Cómo la usan?

Modalidad

- 6 clases presenciales de 1 hora y media cada una
- Teoría y práctica
- La idea es que durante la práctica desarrollen un proyecto web, y que en la clase 5 puedan mostrar lo que lograron
- Puede ser un juego, o un proyecto más serio que les sirva para su trabajo o estudio

Las 6 sesiones

-
- 01 Fundamentos y Vibe Coding
 - 02 Planificación y Revisión
 - 03 Herramientas (MCP) y Habilidades (Skills)
 - 04 Ingeniería de Contexto y Documentación
 - 05 Componentes de un Agente de Código
 - 06 Modelos de Pesos Abiertos y CCAD — UNC
-

Sobre el curso

- Está basado en nuestra experiencia en la industria.
- Usamos coding agents a diario en nuestro trabajo.
- Aprovechen y hagan preguntas !

02

Fundamentos de los LLMs

Contexto histórico

1950



Test de Turing

la pregunta original

2017



Paper Transformers

la arquitectura

HOY



IA generativa

uso masivo

Redes neuronales que pueden generar texto a partir de una secuencia: la idea es vieja, la arquitectura que la hizo funcionar fueron los Transformers.

Por qué recién hoy

Capacidad de cómputo

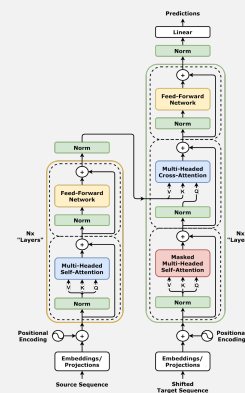
Entrenar estos modelos requiere una escala de hardware que pocos tienen disponible.



Clúster de alta densidad de servidores de infraestructura de IA en un centro de datos de Microsoft.

Avance científico

El paper de los Transformers dio la arquitectura: redes neuronales que pueden generar texto a partir de una secuencia, y que escalan bien con más datos y más cómputo.



Qué significa GPT

Generative

Genera texto.

Pretrained

Preentrenado con datasets
inmensos — en la práctica,
todo internet.

Transformers

La arquitectura de red neuronal
que está debajo de todo esto.

Las tres etapas del entrenamiento

01

Pretraining

Internet crudo. Se genera un **base model**, que predice el próximo token. Del orden de mil GPUs y meses de entrenamiento.

02

Supervised Fine-Tuning

Empieza del base model y agrega respuestas de asistente: pares de instrucción y respuesta ideal escritos por personas expertas en distintos dominios. Se genera un **SFT model**.

03

RLHF

Reinforcement learning a partir de humanos que eligen qué respuesta prefieren. Se entrena un modelo que aprende esas **preferencias**.

El base model completa texto

ENTRADA

Paris is a city in _____

QUÉ HACE

Predice el próximo token, uno por vez, según qué tan probable es que siga.

PALABRA (Token)	PROBABILIDAD
France	17%
and	15%
the	9%
Logan	7%
which	4%
Henry	3%
northern	3%
central	2%
northeastern	2%
Paris	1%
Texas	1%

No responde preguntas: continúa el texto que le diste. Es un predictor del próximo token.

De completar a conversar

SYSTEM

What follows is a conversation between a user and a helpful assistant.

USER

Give me some ideas for what to do when visiting Santiago.

ASSISTANT

Sure, there are _____

Sigue completando texto. Lo que cambia es el formato: una conversación con roles, donde al modelo le toca escribir el turno del asistente.

Alucinaciones

- Cuando un modelo genera, una respuesta que no es verdadera.
- Los hechos arbitrarios y de baja frecuencia — una fecha de nacimiento, por ejemplo — son difíciles de predecir a partir de patrones.
- El modelo no distingue entre lo que sabe y lo que completa: para él las dos cosas son la misma operación.

RAG (como solución a Alucinaciones)

Retrieval Augmented Generation

Recuperar contexto para aumentar el prompt: en vez de confiar en lo que el modelo "aprendió", se le pasa el material relevante junto con la pregunta.

De dónde sale ese contexto

Búsqueda en el código, en la documentación, en la web. El modelo responde sobre lo que tiene adelante, no sobre lo que cree recordar.

03

Cómo se usa un modelo

La ventana de contexto

CONVERSACIÓN

System	User	Assistant	...
--------	------	-----------	-----

Una conversación es una lista de turnos que se manda completa en cada llamada. Todo lo que el modelo tiene en cuenta está ahí adentro, y ese espacio es finito.

CONTEXT ROT

A medida que la ventana se llena, el rendimiento del modelo se degrada: le cuesta más encontrar y usar la información relevante, aunque todavía entre en el contexto.

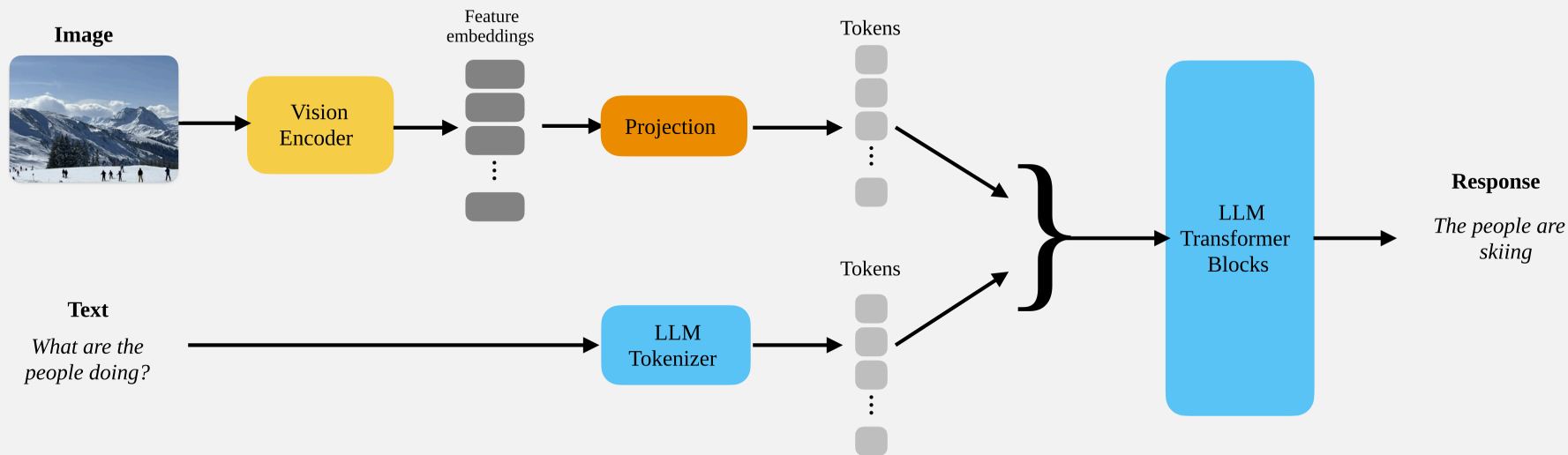
Tools

CONVERSACIÓN CON HERRAMIENTAS

System	User	Tool call	Tool result	Assistant	...
--------	------	-----------	-------------	-----------	-----

- Las herramientas se describen con un JSON Schema: nombre, parámetros, para qué sirven.
- El modelo no ejecuta nada: emite una llamada a función y el resultado vuelve al contexto como un turno más.

Modalidades



en.wikipedia.org/wiki/Vision-language_model

Las dos secuencias de tokens se concatenan y entran juntas a los bloques transformer.

Qué mirar al momento de elegir un modelo

- Context window size — cuántos tokens entran en la ventana de contexto.
- Reasoning — si genera tokens de "razonamiento" antes de responder y cuántos.
- Knowledge cutoff — hasta qué fecha llega lo que aprendió, y qué librerías conoce.
- Caché — si cobra menos por el contexto que ya vio.
- Function calling — si sabe usar herramientas, y qué tan bien.
- Structured output — si puede devolver JSON que cumpla un esquema.

Comparación de modelos de OpenAI: developers.openai.com/api/docs/models/compare

Quién hace los modelos

LABORATORIO	FAMILIA	ORIGEN
OpenAI	GPT	Estados Unidos
Anthropic	Claude — Opus, Sonnet, Haiku	Estados Unidos
Google	Gemini	Estados Unidos
Alibaba	Qwen	China
Moonshot AI	Kimi	China
Z.ai	GLM	China
Mistral	Mistral	Europa

Cuánto cuesta usarlos

Por API

Se paga por token, de entrada y de salida, y el precio cambia según el modelo. Cada proveedor publica su página de comparación de modelos y precios.

Por suscripción

Todos ofrecen planes mensuales. Para desarrollo agéntico suelen ser lo más conveniente: el consumo de tokens es alto y constante.

04

Agentes

“Un agente LLM ejecuta herramientas
en un bucle para lograr un objetivo.”

— Simon Willison

Coding agents

- Agentes para programar: pueden modificar y ejecutar código.
- Sus herramientas son las que usamos para programar — leer, escribir, editar archivos, ejecutar comandos.
- Es la herramienta que vamos a usar en la práctica de las seis clases.

Harness

01

Administra tools

Decide qué herramientas expone al modelo y ejecuta las que el modelo pide.

02

Contexto

Arma y mantiene la ventana: qué entra, qué se resume, qué se descarta.

03

Ejecución del loop

Repite el ciclo de llamada y resultado hasta que el objetivo se cumple.

Por entorno

WEB

Lovable / v0 — más enfocados en vibe coding. Claude Code web.

ESCRITORIO

Claude Desktop

CLI

Claude Code / Codex / OpenCode / **Pi**

Línea de tiempo hasta Coding Agents

2017 *Attention Is All You Need* — el paper que introdujo el Transformer, la arquitectura que hizo posibles los LLMs modernos.

2022 *GitHub Copilot* — junio. Autocompletado de código en el editor, como producto comercial.
ChatGPT — noviembre. Modelo conversacional general, adoptado para escribir, depurar y explicar código.

2023 *Cursor* — un fork de VS Code diseñado desde cero para integrar un LLM, con indexado del código.
Copilot Chat — chat al costado del editor y edición conversacional en línea.

Histórica

2024

Cursor Composer — razonamiento multi-archivo: crear y editar un código base entero desde instrucciones de alto nivel.

Windsurf — un IDE nativo de IA construido alrededor de flujos agénticos sobre todo el workspace.

2025

Claude Code — un agente en la terminal: ejecuta comandos, busca en el repositorio, edita archivos, corre tests y arregla el build.

2025 — 2026

OpenCode · Codex CLI · Gemini CLI · GLM Coding · Kimi CLI · Pi

[coderabbit.ai — A very brief history of AI coding](#)

05

Vibe coding

“There's a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.”

— Andrej Karpathy, febrero 2025 · x.com/karpathy

Vibe coding como videojuego

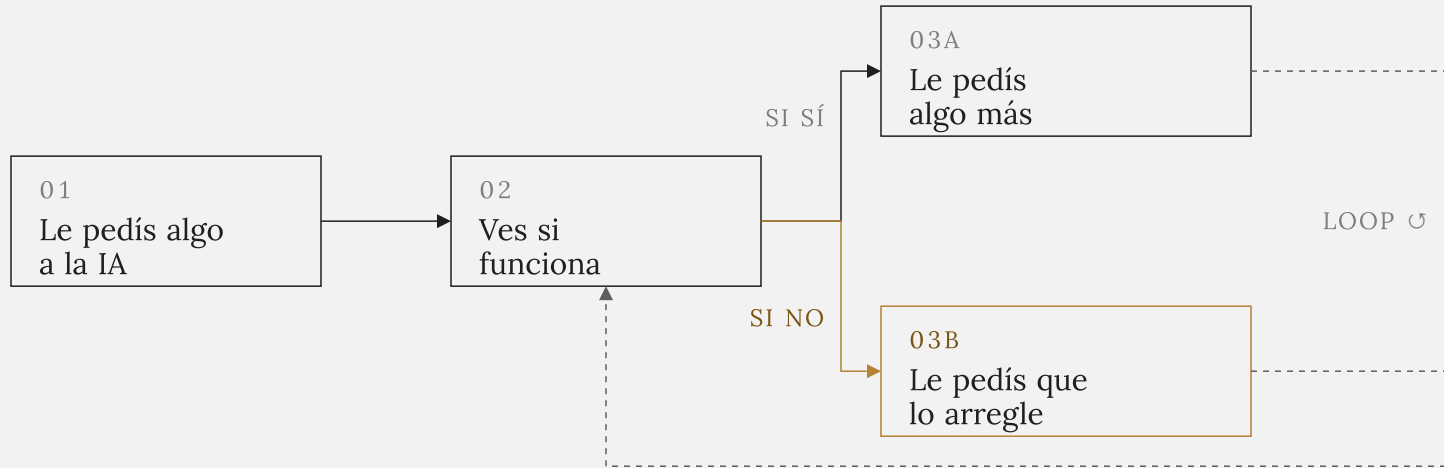
“Vibe coding is more addictive than any video game ever made (if you know what you want to build).”

— Naval Ravikant

- Divertido.
- Feedback y recompensas instantáneas.
- Productivo, constructivo: queda algo hecho.

x.com/naval

El loop de vibe coding.



Tres deudas que se acumulan.

A — DEUDA DE COMPRENSIÓN

No siempre entendés cómo funciona lo que se generó.

Con el tiempo se acumula.

Cada vez entendés menos tu propio proyecto.

B — CÓDIGO FRÁGIL

Anda para el caso feliz. Se rompe en cuanto cambiás algo.

Cada fix agrega complejidad en vez de reducirla.

C — FALTA DE DISEÑO Y ARQUITECTURA

Cada feature se agrega sin pensar cómo encaja con el resto.

Funciona pero no hay un plan. Cada cambio futuro se vuelve más difícil porque no hay estructura debajo.

→ La *deuda de comprensión* es deuda cognitiva; el *código frágil* y *falta de estructura* es deuda técnica.

La IA falla convincentemente.

A veces ni se nota que se equivocó — y eso es lo peligroso.

01 — FALSA CONFIANZA

“Parece que está bien, pero no lo está.”

"Fixed the login issue" — sigue fallando. Te dice que está listo, lo explica con confianza, y no lo está.

03 — ERRORES EN CADENA

Un problema se vuelve varios.

Asume algo incorrecto → construye encima → parece funcionar → pedís un cambio → choca con la suposición → hay que deshacer varias capas.

02 — SIEMPRE TE DA LA RAZÓN

Sycophancy: nunca te contradice.

Le preguntás si está bien y te dice que sí. Le proponés una idea mala y te la celebra. *No es un revisor* — es un asistente entusiasta.

04 — FALTA DE JUICIO

Hace de más, o hace de menos.

“Un botón para guardar” → te lo entrega con validación, toast y logging que no pediste. O al revés: el botón está, pero adentro está vacío. *No mide qué corresponde.*

Vibe coding no es “algo malo”

- Para prototipos y código descartable funciona.
- También para aplicaciones personales: surge la idea de “App Store personal”.

Un caso de Filadd

LA HERRAMIENTA El equipo de Recursos Humanos hizo una plataforma de gestión de objetivos personales: cada empleado entraba y hacía seguimiento de los suyos.

EL PROBLEMA Inspeccionando el network de la página se veía que el frontend accedía a los objetivos de todas las personas, y filtraba el usuario actual del lado del cliente en vez de en el servidor.

EL DESENLACE Se detectó antes de disponibilizar la herramienta. Pero vulnerabilidades de seguridad como esta son un problema de las apps vibecodeadas.

Algunas frases de cierre

- Como desarrolladores, la **responsabilidad** del software sigue siendo nuestra, no de la IA. Si algo no funciona, sea vibecodeado o con Ingenieria Agentica en producción, tenemos que hacernos cargo y **entender** el software producido.
- “Don't outsource the learning” — Addy Osmani · addyosmani.com/blog/dont-outsource-learning

Algunas frases de cierre



x.com/karpathy/status/2049907410303865030

06

Práctica

Demo de Pi

- Web: <https://pi.dev/>
- Ejecutar prompts que cree, edite y ejecute un archivo
- Mostrar tool calls
- Mostrar % de uso de la ventana de contexto y tokens de input/output
- Comandos: `/session`, `/tree`, `/new`
- Cerrar pi con `Ctrl-C` y volver a la sesión con `pi --session <ID>` o `pi --resume`

Inicio de su proyecto

- Setup de Pi: pi.dev/docs/latest/quickstart
- Crear un repo de Git del proyecto que vayan a trabajar durante el curso (versionen los cambios)
- Seguí las siguientes reglas para que sea vibecoding:

1. Hablale al agente.

Describí lo que querés en lenguaje natural.

3. Si algo se rompe, describí el síntoma, no lo diagnostiques.

"El botón no hace nada" está bien. "Falta el event listener en `App.jsx`" está prohibido — para saber eso tendrías que haber leído el código.

2. No abras los archivos.

Ni en el IDE, ni con `cat`, ni con `git diff`. No leas lo que escribió.

4. Juzgá solo por el output.

¿Se ve bien? ¿Corre? ¿Hace lo que pediste? Con eso decidís si seguí o iterás.

Al final de la clase recién pueden ver los archivos y el código que generaron

Si van a levantar un server de desarrollo, pueden instalar el paquete [@aliou/pi-processes](https://github.com/aliou/pi-processes)